

Pyrrmit – technical report

Data, Swedish legal context, pipeline and evaluation methodology. Companion to the benchmark report.
Generated on 2026-10-10; figures are read from the data and result files at generation time.

- | | |
|---|--|
| 1. Purpose of the system | 6. Retrieval evaluation |
| 2. The Swedish planning system and where Pyrrmit fits | 7. Agent evaluation: methodology in detail |
| 3. Data sources: what, why, how | 8. Results summary |
| 4. Keeping the data current | 9. Limitations and roadmap |
| 5. Pipeline, with code references | Appendix: commands and files |

1. Purpose of the system

Pyrrmit answers, in English, whether a building or land project is feasible in a given Swedish municipality and how to get it approved: which rules decide the case (national law, the municipality's plans and local guidance), what permits are needed, and what remains uncertain. Every rule, number or deadline in an answer must cite a passage the system actually read; citations open the original document at that passage. It is a **pre-application screening** tool: it does not replace the municipality's decision, and the answers say what must be checked.

2. The Swedish planning system and where Pyrrmit fits

2.1 Who decides what

- **Municipal planning monopoly.** Planning the use of land and water is a municipal matter (PBL 1 kap. 2 §). The municipality's building committee (byggnadsnämnd) decides on permits; its planning bodies adopt plans.
- **Comprehensive plan (översiktsplan, PBL 3 kap.):** covers the whole municipality, states intentions (where to build, what to protect). Not binding, but it guides permit and planning decisions.
- **Detailed plan (detaljplan, PBL 4 kap.):** binding for an area: land use (residential, roads, nature...) and property provisions (building area, ridge height, storeys, minimum plot size, land that may not be built on, design, implementation period). It consists of a *plankarta* (map with codes and provisions) and a *planbeskrivning* (explanatory description). Plans go through consultation (samråd), review (granskning), adoption (antagande) and gain legal force (laga kraft). Area regulations (områdesbestämmelser) are a lighter binding instrument.
- **Permits (PBL 9 kap.):** building permit (bygglov), demolition permit (rivningslov), site permit (marklov); some measures only need a notification (anmälan) or are exempt (e.g. Attefall measures). Inside a detailed plan the project must comply with the plan (small deviations can be accepted); outside, the site is assessed against the general suitability requirements of PBL 2 kap. An advance ruling (förhandsbesked) can settle the location question first. **Note:** PBL chapter 9 was renumbered by SFS 2025:974 (bygglov conditions now 9 kap. 56–57 §, small deviations 60–61 §, advance ruling 74 §); case law and most guidance still use the old numbers.
- **Environmental Code (miljöbalken)** constraints that often decide feasibility: shoreline protection (strandskydd, MB 7 kap.; building needs an exemption with a special reason), productive agricultural land (MB 3 kap. 4 §: only for essential public interests), areas of national interest (MB 3–4 kap.).
- **Water and sewage:** municipal water and sewage areas (LAV); a private sewage system needs a permit or notification to the municipal environmental committee (förordningen om miljöfarlig verksamhet och hälsoskydd, 13–14 §§).
- **Property formation** (fastighetsbildning) is decided by Lantmäteriet under the Real Property Formation Act; detailed plans can set minimum plot sizes.
- **Appeals:** building committee decisions are appealed to the county administrative board (länsstyrelsen), then to a Land and Environment Court (mark- och miljödomstol), then to the Land and Environment Court of

Appeal (Mark- och miljööverdomstolen, MÖD), which needs leave to appeal and sets precedents. Decisions to adopt a detailed plan are appealed directly to the Land and Environment Court.

2.2 Where Pyrrmit fits

Question in a project	Legal basis	What Pyrrmit reads
Is my site inside a detailed plan, and what does it allow?	PBL 4 kap., plan provisions	Plan registry, plan maps and descriptions crawled from the municipality (tools <code>plans</code> , <code>plan_rules</code> , <code>read</code>)
Do I need a permit, a notification, or nothing?	PBL 9 kap., PBF	Statute text (<code>law_section</code>), municipal permit pages
Can I build outside a plan / get an advance ruling?	PBL 2 kap., 9 kap.	Statute text, comprehensive plan, municipal guidance
Shoreline, farmland, national interests?	MB 3, 4, 7 kap.	Statute text, comprehensive plan, municipal pages on strandskydd
Water and sewage, fees, timelines?	LAV, FMH, municipal fee schedules (taxa)	Municipal pages and PDFs (sewage guidance, taxa)

The system mirrors how a planning officer screens a case: identify the municipality and the site, check the plan situation, find the national rules that apply, then the local documents. It stops before what needs official data it does not have yet (which plan covers a given property, maps of protected areas) and says so in the answer.

3. Data sources: what, why, how

Source	What we take	Why this source	How (code)	Terms
Riksdagen open data (data.riksdagen.se)	15 consolidated statutes (PBL, PBF, MB, JB, FBL, LAV, FMH...)	Official consolidated text with amendment markers; free API	<code>corpus/laws.py</code> (<code>LAWS</code> , <code>main</code>)	Open data, cite Riksdagen
Wikidata (SPARQL)	290 municipalities: official code, county, website; ~10k localities	Stable identifiers and official websites; localities for place → municipality	<code>corpus/municipalities.py</code> (<code>sparql</code> , <code>pick_site</code>)	CC0
Municipal websites	31,086 pages, 7,769 PDFs (18.3 GB): permits, sewage, fees, plans, comprehensive plans	Local rules and detailed plans are published by each municipality; no national full-text source	<code>corpus/crawl.py</code> (<code>content_relevance</code> , <code>crawl</code>)	Public web pages; robots.txt honoured, 1 request/s per host
Local OCR (EasyOCR, GPU)	Text of 777 scanned PDFs (old plan maps)	Old plans have no text layer	<code>corpus/ocr.py</code>	Local processing
Plan registry (derived)	8,258 plans, 1,582 with 16,568 extracted provisions	Agent needs plans as entities with status and rules	<code>corpus/plans.py</code> (<code>plan_key</code> , <code>provisions_of</code> , <code>status_of</code>)	Derived
Case law: MÖD via lagen.nu	746 rulings since 2011, 390 on our topics	Real decisions with facts, outcome and reasons: ground truth for the benchmark	<code>corpus/caselaw.py</code> (<code>parse_case</code> , <code>relevant</code>)	Court rulings are public documents without copyright protection; lagen.nu allows the ruling pages in robots.txt

3.1 Statute versions in the corpus

Statute	Consolidated up to	Downloaded
Anläggningslag (1973:1149)	SFS 2022:362	2026-10-06
Bostadsrättslag (1991:614)	SFS 2026:847	2026-10-06
Expropriationslag (1972:719)	SFS 2019:850	2026-10-06
Fastighetsbildningslag (1970:988)	SFS 2020:921	2026-10-06
Förordning (1998:899) om miljöfarlig verksamhet och hälsoskydd	SFS 2026:1515	2026-10-06
Jordabalk (1970:994)	SFS 2026:1498	2026-10-06
Kulturmiljölag (1988:950)	SFS 2024:813	2026-10-06
Lag (2006:412) om allmänna vattentjänster	SFS 2022:1249	2026-10-06
Lag (1973:1150) om förvaltning av samfälligheter	SFS 2026:846	2026-10-06
Ledningsrättslag (1973:1144)	SFS 2021:742	2026-10-06
Miljöbalk (1998:808)	SFS 2026:1442	2026-10-06
Miljöbedömningsförordning (2017:966)	SFS 2026:706	2026-10-06
Miljöprövningsförordning (2013:251)	SFS 2026:943	2026-10-06
Plan- och byggförordning (2011:338)	SFS 2026:1722	2026-10-06
Plan- och bygglag (2010:900)	SFS 2026:1583	2026-10-06

3.2 Crawl

286 municipalities crawled between 2026-10-06 and 2026-10-07 (82,389 requests). The crawler starts from each official website, follows links scored by topic keywords (building permits, plans, sewage, fees; negative keywords for unrelated sections), keeps pages with a relevance score ≥ 4 (`KEEP_SCORE`), PDFs linked from relevant pages, and caps each municipality (300 pages, 150 PDFs, 40 MB per PDF, 1,500 requests, depth 4, per-section quotas). Text is extracted with PyMuPDF block by block, repeated headers and footers removed (`corpus/extract.py` `page_blocks` , `strip_boilerplate`), each document classified (`doc_type` : detaljplan 9,554, bygglov 5,744, avlopp 5,571, other 5,320, översiktsplan 4,222, taxa 1,435, strandskydd 386).

Corpus served by the app: 348,736 chunks in 32,247 documents, 275 municipalities, 3,651 statute sections. Plan statuses in the registry: `in_force` 4,161, `in_progress` 1,724, `unknown` 1,646, `consultation` 356, `review` 220, `adopted` 151.

3.3 Sources not used yet (and why they matter)

- **Lantmäteriet**: the property register (property designations, boundaries) and national access to digital detailed plans. Needed to answer “which plan covers my property” reliably. Access terms and APIs to be confirmed (some are paid).
- **Svensk författningssamling** ([svenskforfattningssamling.se](https://www.svenskforfattningssamling.se)): the official publication of new acts; useful to detect amendments before the consolidated text is updated.
- **County boards and national agencies**: GIS layers for shoreline protection, national interests, protected nature; Boverket's guidance (PBL kunskapsbanken). Would turn spatial questions (“is my plot within 100 m of the shore?”) into data lookups.
- **Municipal plan services** (WMS / plan registers): many municipalities publish plan boundaries as map services; combined with an address or property, they give the applicable plan.

4. Keeping the data current

4.1 Current state (honest picture)

- Statutes: the consolidated version is visible in the raw text ("t.o.m. SFS ...", table 3.1) but not stored as metadata; texts with two versions of a section (in force now / from a future date) are both indexed.
- Municipal documents: **no fetch date or HTTP validators are stored per page or PDF** (the crawl manifest keeps id, URL, title, score); the only date is the crawl itself (2026-10-06–2026-10-07). Fee schedules change yearly; plan pages change when plans move through the process.
- Plans: status is inferred from page URLs and document text (laga kraft date, "antagandehandling"...); superseded or repealed plans are not tracked.
- Rebuilds are manual: extract → chunk → embed (incremental) → store → plan registry. Chunk ids include a text hash, so unchanged chunks keep their vectors.

4.2 How to guarantee freshness

1. **Provenance on every document:** fetched_at, HTTP ETag / Last-Modified, content hash, source URL, and for statutes the "t.o.m. SFS" version and the date of entry into force of each section version. Shown in the source panel and in the answer ("municipal page fetched on ...").
2. **Scheduled refresh by volatility:** statutes weekly (Riksdagen API, compare the version marker; check svenskforfattningssamling.se for new acts); municipal plan and permit pages weekly, the rest of the crawl monthly, with conditional requests (If-None-Match / If-Modified-Since) and sitemaps so unchanged pages cost one cheap request; case law monthly.
3. **Incremental pipeline:** re-extract and re-chunk only changed documents, embed only new chunk ids, update the store in place (insert / delete rows and vectors) instead of rebuilding; rebuild the plan registry for touched municipalities.
4. **Legal validity:** index each statute section with its validity interval (the consolidated text marks future versions "Träder i kraft l:..." and expiring ones "Upphör att gälla U:..."), answer with the version in force on the question date; track plan status changes (adopted → legal force → amended / repealed).
5. **Prefer authoritative sources** where they exist (Lantmäteriet's digital plans and property data) over crawled copies, and keep the crawl for what has no national source (local guidance, fees).
6. **Monitoring and regression:** after each refresh, a report of changed / failed sources per municipality, and a benchmark run (MÖD sample) to catch quality regressions; alert when a municipality's site structure changes (crawl yields drop).

5. Pipeline, with code references

Stage	What it does	Code
Municipalities	Wikidata SPARQL → codes, websites, localities	<code>corpus/municipalities.py</code>
Statutes	Riksdagen API → raw text per SFS	<code>corpus/laws.py</code>
Crawl	Async, robots.txt, 1 req/s/host, topic scoring, caps, resumable manifests	<code>corpus/crawl.py</code>
OCR	EasyOCR sv+en on GPU, tiled pages, plans first	<code>corpus/ocr.py</code>
Extraction	PyMuPDF blocks, boilerplate removal, doc types, OCR text when needed	<code>corpus/extract.py</code>
Chunking	One chunk per statute section (chapters only on their own line, lettered §, transitional provisions dropped); ~1,500-char document chunks; corpus-wide dedup; ids = hash(doc, index, text)	<code>corpus/chunk.py</code> (<code>law_chunks</code> , <code>document_chunks</code>)
Embeddings	Snowflake arctic-embed-l-v2 (1024 dim) on the GPU, incremental	<code>localrag/index.py</code> , <code>embeddings.py</code>
Disk store	SQLite (chunks, FTS5 BM25 on Swedish stems with pool / municipality filter tokens) + float16 vectors (mmap → GPU)	<code>localrag/store.py</code> (<code>build</code> , <code>DiskStore</code> , <code>DiskRetriever</code>)
Plan registry	Plans, status, provisions tied to chunks	<code>corpus/plans.py</code>
Place → municipality	Wikidata names and localities U corpus place names	<code>localrag/gazetteer.py</code>
Agent tools	<code>find_kommun</code> , <code>search</code> (hybrid RRF), <code>grep</code> , <code>list_documents</code> , <code>read</code> , <code>law_section</code> , <code>plans</code> , <code>plan_rules</code> , <code>view_pdf_page</code> ; every output tagged with <code>[c:chunk_id]</code>	<code>agentic/tools.py</code> (<code>CorpusTools</code> , <code>TOOL_SCHEMAS</code>)
Agent loop	Responses API tool loop: exploration turns at <code>explore_effort</code> , parallel tool calls, <code>answer_ready</code> , streamed final answer at <code>effort</code> , step budget	<code>agentic/agent.py</code> (<code>run_agent</code> , <code>SYSTEM_PROMPT</code>)
Chat integration	Background thread, SSE events, live citation numbering, saved trace	<code>agentic/service.py</code> (<code>stream_agent</code> , <code>CitationNumberer</code>), <code>routers/chat.py</code>
Source viewer	Cited chunk → PDF page with highlight, or URL text fragment	<code>routers/docs.py</code> , <code>frontend/components/SourceViewer.tsx</code>

Grounding by construction. Tool outputs carry chunk ids; the answer format requires a `[c:id]` for every rule, number, fee or deadline, and forbids stating what was not read in a tool result. The chat replaces ids by numbers while streaming and attaches, for each number, the cited chunks (pipeline event `citations`).

6. Retrieval evaluation

`localrag/eval.py` on `localrag/questions.jsonl`: 45 use-case questions (permits, plans, sewage, fees, heights...), each labelled with the passages that answer it (exact excerpts of municipal documents or statute sections). Metrics per question: `recall@k` = share of the expected passages found in the top k; `complete@5` = all expected passages in the top 5; `MRR` = $1 / \text{rank of the first expected passage}$. The municipality filter is detected from the question. Setups: *dense-en* / *dense-sv* = embedding of the English question / of a Swedish rewrite; *bm25* = BM25 on Swedish tokens; *hybrid* = reciprocal-rank fusion; *+rerank* = cross-encoder bge-reranker-v2-m3.

Model	Setup	R@1	R@5	R@10	Complete@5	MRR
st-arctic-l-v2	dense-sv	0.82	0.92	0.93	0.87	0.86
st-bge-m3	dense-sv	0.78	0.91	0.91	0.84	0.83
or-qwen3-8b	dense-sv	0.71	0.91	0.94	0.84	0.78
st-kblab-swedish	dense-sv	0.51	0.70	0.86	0.60	0.61
st-bge-m3	dense-en	0.32	0.63	0.79	0.53	0.48
or-qwen3-8b	dense-en	0.50	0.78	0.88	0.71	0.62
st-arctic-l-v2	hybrid-sv	0.69	0.87	0.89	0.78	0.75
st-arctic-l-v2	dense-sv +rerank	0.73	0.88	0.93	0.78	0.81
BM25	bm25-sv	0.51	0.76	0.80	0.62	0.62
BM25	bm25-sv-stem	0.51	0.69	0.81	0.61	0.60
BM25	bm25-en	0.00	0.04	0.06	0.04	0.02

Findings: querying in Swedish is decisive (+13 to +28 points of recall@5 over the English query, depending on the model); local open models match API models (arctic-embed-l-v2: 0.92 recall@5); BM25 on English queries is useless on a Swedish corpus; neither BM25 fusion nor the reranker beat dense retrieval on the Swedish query. The agent still uses hybrid search because exact wording (plan names, "20 meter") matters inside its tool calls.

7. Agent evaluation: methodology in detail

7.1 Benchmark sets

- **14 case studies** (`agentic/cases.jsonl`), hand-written. Fields: `question`, `kommun` (codes), `verdicts` (accepted), `key_points`, `evidence` (matchers), `source`.
- **260 MÖD cases** (`agentic/cases_caselaw.jsonl`) generated by `agentic/cases_from_caselaw.py` from 390 topical rulings: one LLM call per ruling with the prompt below; drafts cached in `data/corpus/raw/caselaw/case_drafts.jsonl`; rejected when not usable (procedure, sanctions, standing) or when the question mentions the court (`LEAK` regex); `kommun` name → code (`kommun_codes`, genitive forms); statutes → evidence matchers (`evidence_for`).
- **Sampling:** `sample_cases` in `agentic/bench.py` draws N cases with a fixed seed, round-robin over three strata (negative = refused, positive = granted, open = remitted / fact-dependent), labelled cases first. The report uses N = 50.

Case-generation prompt (`agentic/cases_from_caselaw.py` INSTRUCTIONS):

```

You build test cases for an assistant that tells people whether a building or land project is
feasible in Sweden. You get a ruling of Mark- och miljööverdomstolen (MÖD). Return JSON only:
{
  "usable": true|false,    // false if the ruling is not about whether a concrete project/measure may go ahead
                          // (pure procedure, fees, sanctions, appeal standing, environmental permits for industry).
                          // A förhandsbesked, bygglov, rivningslov, marklov, strandskyddsdispens, change of use or
plan
                          // IS usable whatever the outcome, also when MÖD remitted it after stating the decisive
legal
                          // test (then verdicts ["unclear"] or ["conditional","unclear"]).
  "usable_reason": "<one sentence>",
  "kommun": "<kommun name, e.g. Varberg>",
  "question": "<English, 2-5 sentences, written by the applicant before applying: the project and the facts that
the
the
the
detaljplan/översiktsplan says, shoreline, farmland...). Never the outcome, the court's assessment or
the
case number. Do not name the property designation; you may name the kommun and the area.>",
  "outcome": "<one sentence, what MÖD decided>",
  "verdicts": ["feasible"|"conditional"|"not_feasible"|"unclear", ...], // acceptable verdicts for an assistant
                          // answering the question: permit granted -> ["feasible","conditional"]; refused ->
                          // ["not_feasible"] (add "unclear" only if the facts in the question leave it open)
  "key_points": ["<3-5 English points an expert answer must make, each naming the rule, e.g. 'Outside a detaljplan
a new house needs bygglov and a localisation assessment under PBL 2 kap. (PBL 9 kap. 31 §)'>"],
  "statutes": [{"law": "<one of PBL, PBF, MB, JB, FBL, AL, LAV, FMH, KML>", "section": "<as cited in the ruling,
e.g. 9 kap. 31 b §>",
    "quote": "<5-12 consecutive words of the provision's own wording, copied exactly as the ruling
quotes
the statute (not the court's paraphrase), e.g. 'avvikelsen är förenlig med detaljplanens
syfte'>"}]
                          // 1-4 provisions decisive for the outcome (PBL 2010:900, MB 1998:808, ...). Leave out
                          // provisions of the old PBL (1987:10, ÄPBL) and ones the ruling does not quote.
}

```

7.2 Evidence labels and matching

A matcher is a dict checked against every chunk the agent was shown (`localrag/eval.py matches`): `law` (substring of the statute title) + `section` or `contains` (verbatim phrase, whitespace- and case-normalised), `kommun_code` + `contains` / `title`, or `any` (alternatives). For MÖD cases, the generator asks for 5–12 words of the statute as quoted in the ruling, then keeps the **longest run of ≥ 5 consecutive words found in the current statute text** (`longest_phrase`): this survives renumbering and small amendments (“tas i anspråk” → “tas i anspråk eller inreds”). Without a match, a section number is used only outside the renumbered PBL 9 kap.; otherwise the case has no evidence label. `check_labels` refuses to run a benchmark whose labels match no chunk.

7.3 Metrics

Metric	Formula	Code
verdict_ok	judge's verdict ∈ accepted verdicts	<code>judge, run_case</code>
points	$\sum \text{judge scores (0/1/2 per key point)} / (2 \times \text{number of key points})$	<code>run_case</code>
evidence	share of matchers satisfied by at least one chunk the agent saw (tool outputs), no LLM	<code>evidence_recall</code>
grounded	supported claims / claims, second judge with the full text of the cited chunks	<code>agentic/grounding.py</code>
unsupported	claims in neither the reference nor the cited sources (first judge)	<code>judge</code>
seconds, first_token_s, cost	wall time; time to the first streamed answer token; OpenRouter <code>usage.cost</code> summed over calls	<code>agentic/agent.py</code> <code>AgentRun</code>

The judge receives the question, the key points, the accepted verdicts, the answer and up to 15 cited chunks (1,500 characters each). Prompt (`agentic/bench.py` `JUDGE_INSTRUCTIONS`):

```
You grade answers of a Swedish planning assistant against a reference.
Return JSON only: {"verdict": "<feasible|conditional|not_feasible|unclear>", "points": [<0|1|2> per key point, in
order], "unsupported_claims": <int>, "comment": "<one sentence>"}.
- verdict: how the answer classifies the project (Feasible -> feasible, Feasible with conditions -> conditional,
Not feasible as described -> not_feasible, Unclear -> unclear).
- points: 2 = the key point is clearly stated, 1 = partially / vaguely, 0 = missing or contradicted.
- unsupported_claims: number of specific rules/numbers/facts in the answer that are stated neither in the reference
nor
in the cited sources given below, or that contradict them. A claim the cited sources state is supported even when
the reference does not mention it (the reference is not exhaustive).
```

Grounding prompt (`agentic/grounding.py` `INSTRUCTIONS`):

```
You check whether an answer is grounded in its cited sources.
List the specific factual claims of the answer (rules, numbers, deadlines, fees, document facts, legal references);
ignore generic advice. For each, decide if the cited sources support it.
Return JSON only: {"claims": <int>, "supported": <int>, "unsupported_examples": [<"<short claim>", ...up to 3]>}.
```

7.4 Protocol

- Corpus: the full disk store (`--store disk`), so cases from any municipality are answerable.
- Same model for agent and judges (GPT-6 Luna via OpenRouter); judge effort medium.
- Repeats (`--repeats`) and merged files give 2–4 passes per configuration; per-pass means measure noise.
- Cases run in parallel (`--workers 3`): latencies include some contention.
- Results are written after every case (`data/agentic/bench-*.json`, one row per answer with the full answer, tool steps and judge comment); `--resume` completes an interrupted run.
- Reports: `agentic/report.py` (summary) and this file, `agentic/report_technical.py`.

8. Results summary

MÖD sample (50 cases); details and charts in the benchmark report.

Configuration	n	Verdict	Key points	Evidence	Grounding	Tool calls	Latency	1st token	Cost / question
Agent, exploration low / answer medium	100	0.75	0.43	0.53	0.69	12.3	26 s	19 s	\$0.0036
Agent, exploration medium / answer medium	200	0.70	0.48	0.70	0.79	26.4	43 s	35 s	\$0.0086
Agent, exploration medium / answer high	150	0.71	0.48	0.74	0.79	26.7	51 s	42 s	\$0.0092

Configuration	Project refused (correct verdict)	Project granted	Open outcome	On refusals, answers "with conditions"
Agent, exploration low / answer medium	0.59 (32)	0.74 (34)	0.91 (34)	0.28
Agent, exploration medium / answer medium	0.39 (64)	0.79 (68)	0.90 (68)	0.44
Agent, exploration medium / answer high	0.50 (48)	0.73 (51)	0.90 (51)	0.38

Exploration effort is the setting that matters (evidence 0.53 → 0.70, grounding 0.69 → 0.79); a high-effort answer adds nothing; the agent under-calls “not feasible” on refused projects (40–60 %).

9. Limitations and roadmap

- **Evaluation:** LLM judge = agent model (possible leniency); key points written by an LLM; no human review yet. Next: expert review of a sample, a second judge model, inter-judge agreement.
- **Data:** coverage depends on each municipality’s website; no property → plan link; no spatial layers; no provenance dates (section 4). Next: provenance metadata, scheduled incremental refresh, Lantmäteriet and county GIS data.
- **Plans:** provisions are extracted as text; which map area carries which code needs a vision model on the plan map.
- **Agent:** verdict calibration on refusals (prompt), default exploration effort medium, a fast low-effort mode.

Appendix: commands and files

```
cd backend
python -m corpus.municipalities && python -m corpus.laws && python -m corpus.crawl
python -m corpus.ocr && python -m corpus.extract && python -m corpus.chunk
python -m localrag.index --model st-arctic-l-v2 && python -m localrag.store --model st-arctic-l-v2 && python -m
corpus.plans
python -m localrag.eval --models st-arctic-l-v2
python -m corpus.caselaw && python -m agentic.cases_from_caselaw
python -m agentic.bench --systems agent --store disk --explore-effort medium --effort medium \
    --cases-file cases_caselaw.jsonl --sample 50 --repeats 2 --workers 3
python -m agentic.grounding data/agentic/bench-....json
python -m agentic.report && python -m agentic.report_technical
```